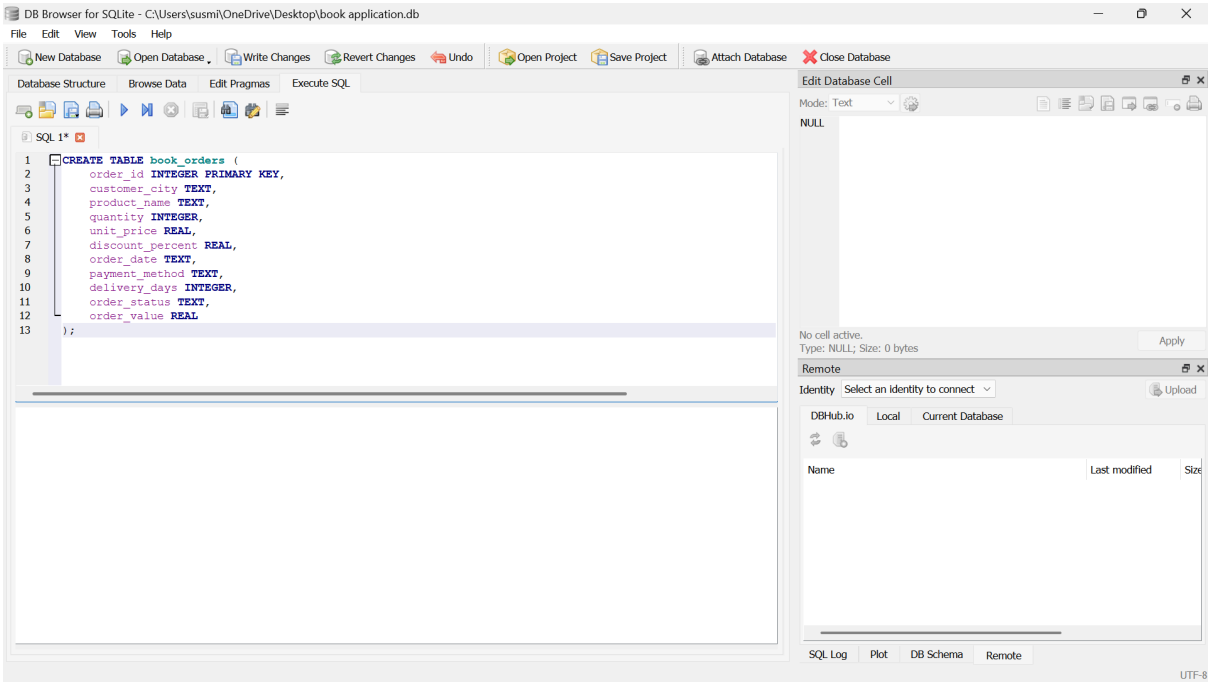
Database operations on Book Orders dataset

Step 1 — Create the table

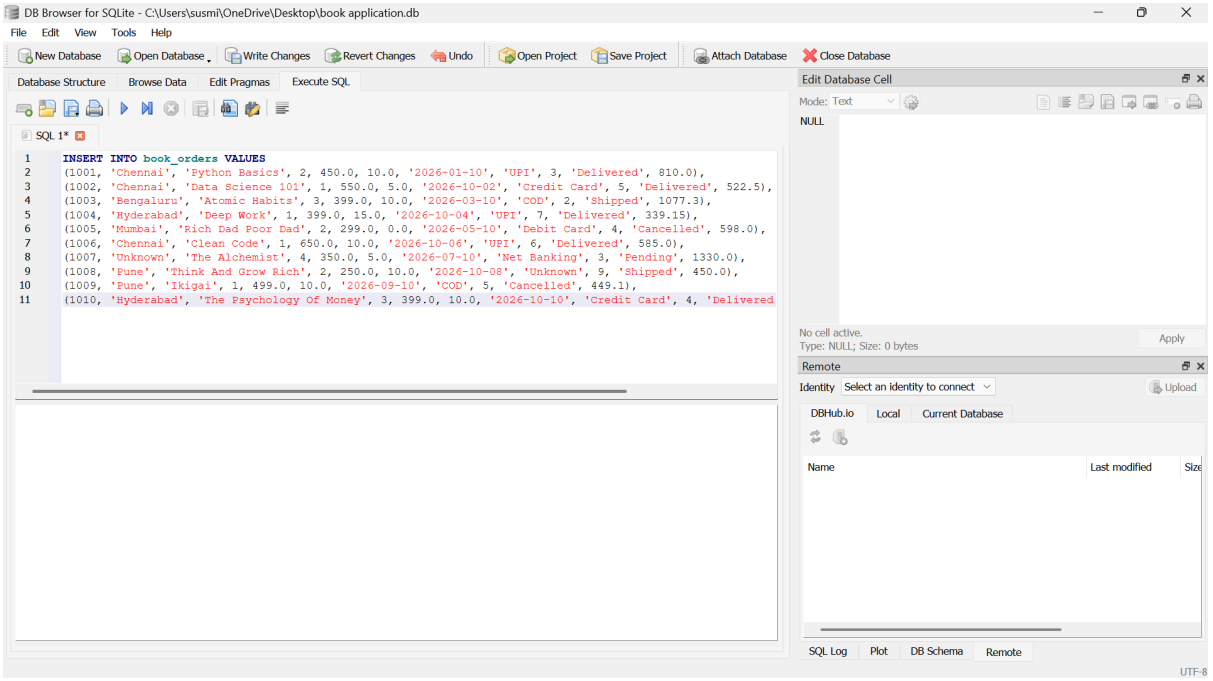
```sql
CREATE TABLE book_orders (
    order_id INTEGER PRIMARY KEY,
    customer_city TEXT,
    product_name TEXT,
    quantity INTEGER,
    unit_price REAL,
    discount_percent REAL,
    order_date TEXT,
    payment_method TEXT,
    delivery_days INTEGER,
    order_status TEXT,
    order_value REAL
);
```



Step 2 — Insert all 10 records

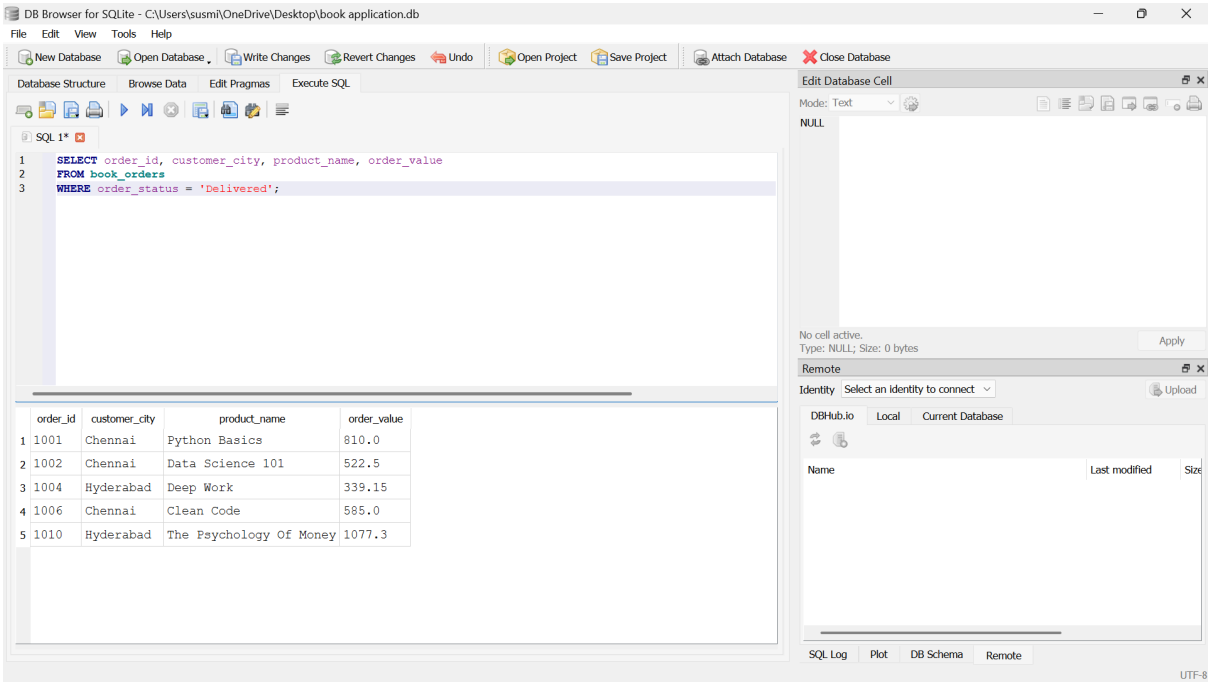
```sql
INSERT INTO book_orders VALUES
(1001, 'Chennai', 'Python Basics', 2, 450.0, 10.0, '2026-01-10', 'UPI', 3,
'Delivered', 810.0),
(1002, 'Chennai', 'Data Science 101', 1, 550.0, 5.0, '2026-10-02', 'Credit
Card', 5, 'Delivered', 522.5),
(1003, 'Bengaluru', 'Atomic Habits', 3, 399.0, 10.0, '2026-03-10', 'COD', 2,
'Shipped', 1077.3),
```

```
(1004, 'Hyderabad', 'Deep Work', 1, 399.0, 15.0, '2026-10-04', 'UPI', 7,
'Delivered', 339.15),
(1005, 'Mumbai', 'Rich Dad Poor Dad', 2, 299.0, 0.0, '2026-05-10', 'Debit
Card', 4, 'Cancelled', 598.0),
(1006, 'Chennai', 'Clean Code', 1, 650.0, 10.0, '2026-10-06', 'UPI', 6,
'Delivered', 585.0),
(1007, 'Unknown', 'The Alchemist', 4, 350.0, 5.0, '2026-07-10', 'Net Banking',
3, 'Pending', 1330.0),
(1008, 'Pune', 'Think And Grow Rich', 2, 250.0, 10.0, '2026-10-08', 'Unknown',
9, 'Shipped', 450.0),
(1009, 'Pune', 'Ikigai', 1, 499.0, 10.0, '2026-09-10', 'COD', 5, 'Cancelled',
449.1),
(1010, 'Hyderabad', 'The Psychology Of Money', 3, 399.0, 10.0, '2026-10-10',
'Credit Card', 4, 'Delivered', 1077.3);
```



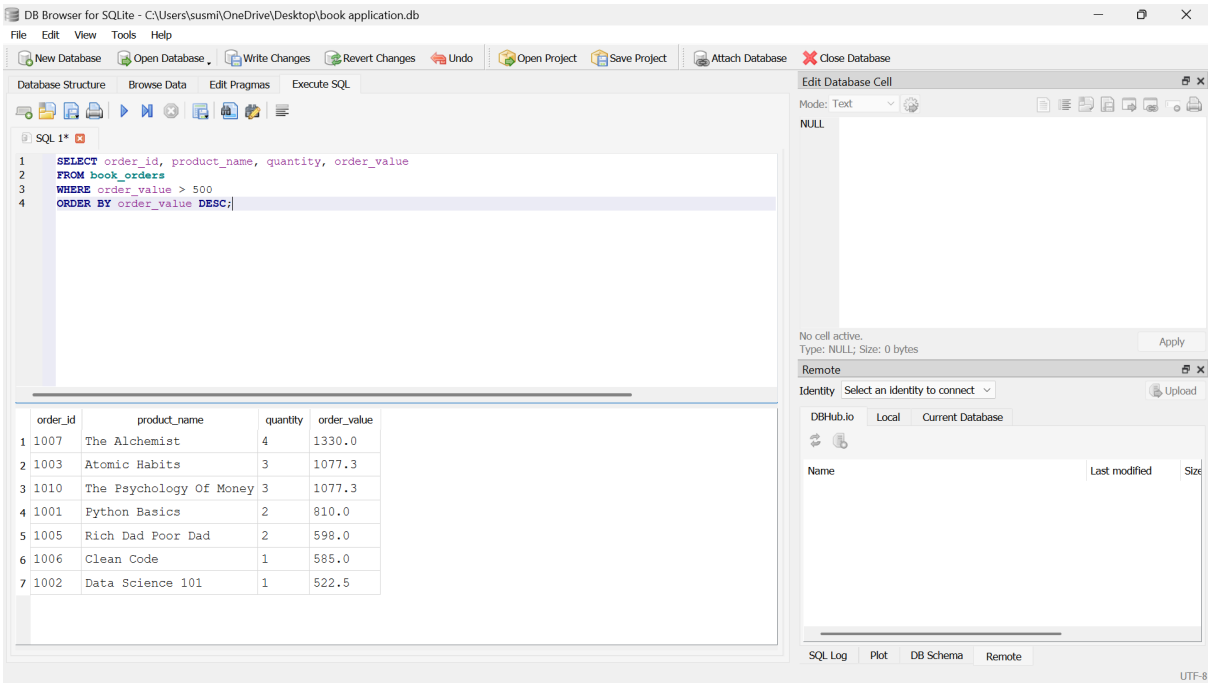
Step 3 — Filter delivered orders

```sql
SELECT order_id, customer_city, product_name, order_value
FROM book_orders
WHERE order_status = 'Delivered';
```



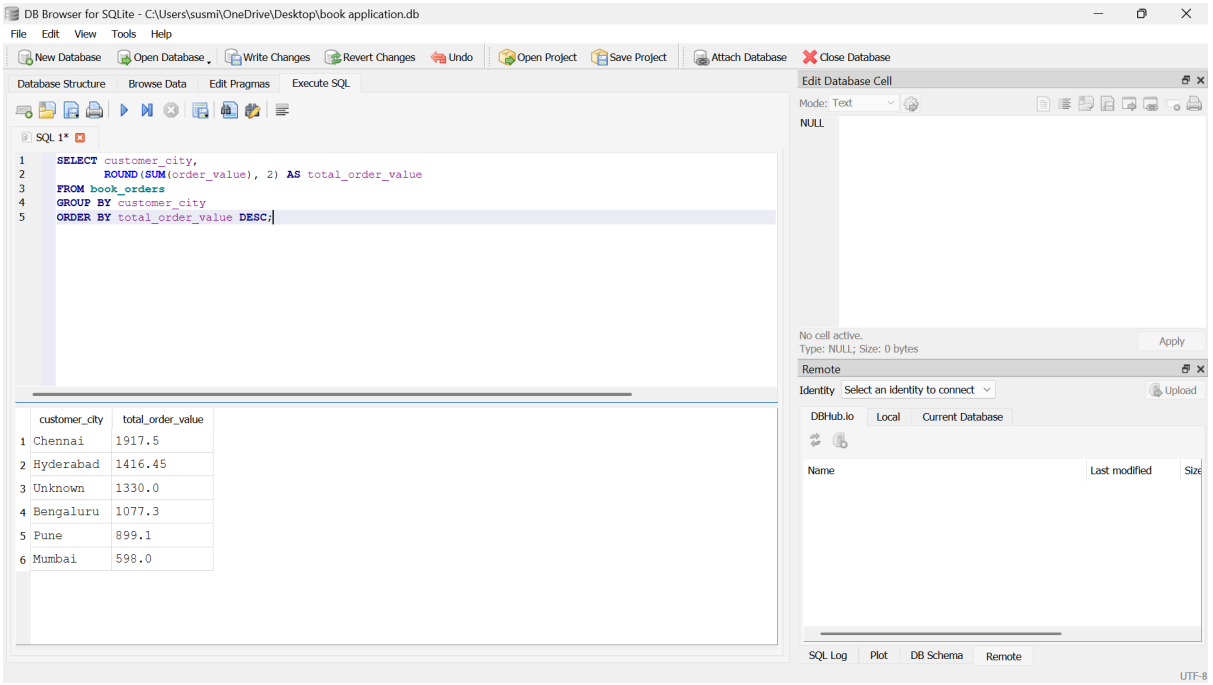
Step 4 — Find orders worth more than ₹500

```sql
SELECT order_id, product_name, quantity, order_value
FROM book_orders
WHERE order_value > 500
ORDER BY order_value DESC;
```



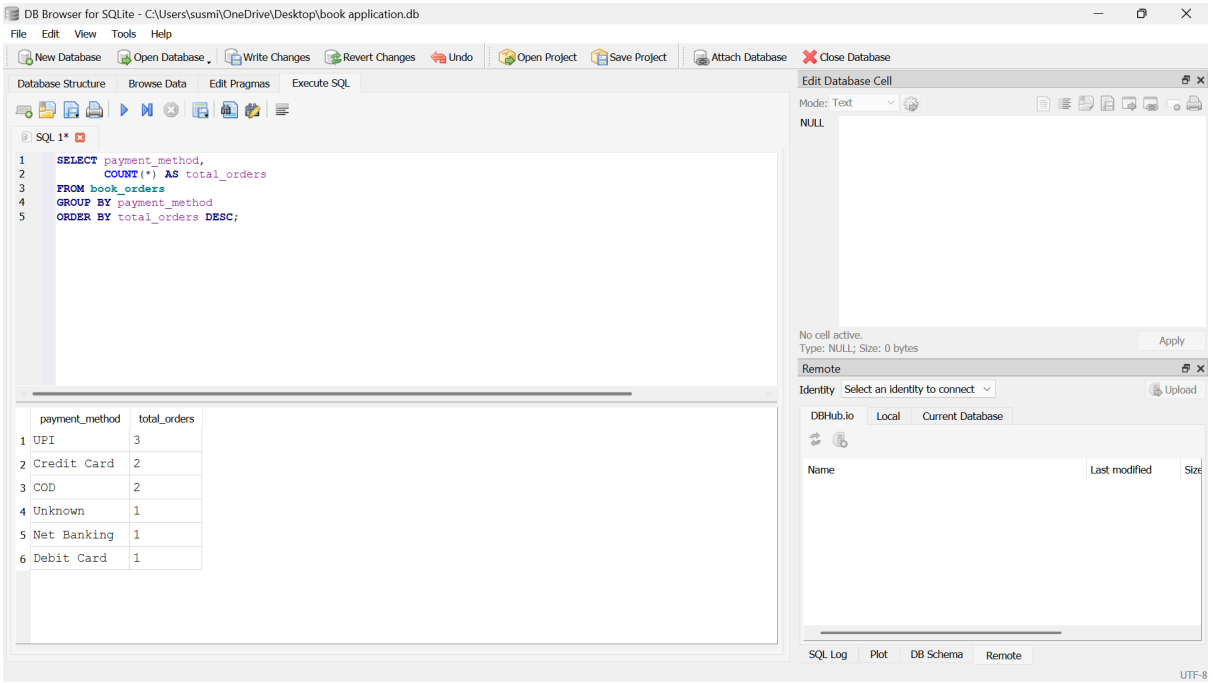
Step 5 – Calculate total order value by city

```sql
SELECT customer_city,
       ROUND(SUM(order_value), 2) AS total_order_value
FROM book_orders
GROUP BY customer_city
ORDER BY total_order_value DESC;
```



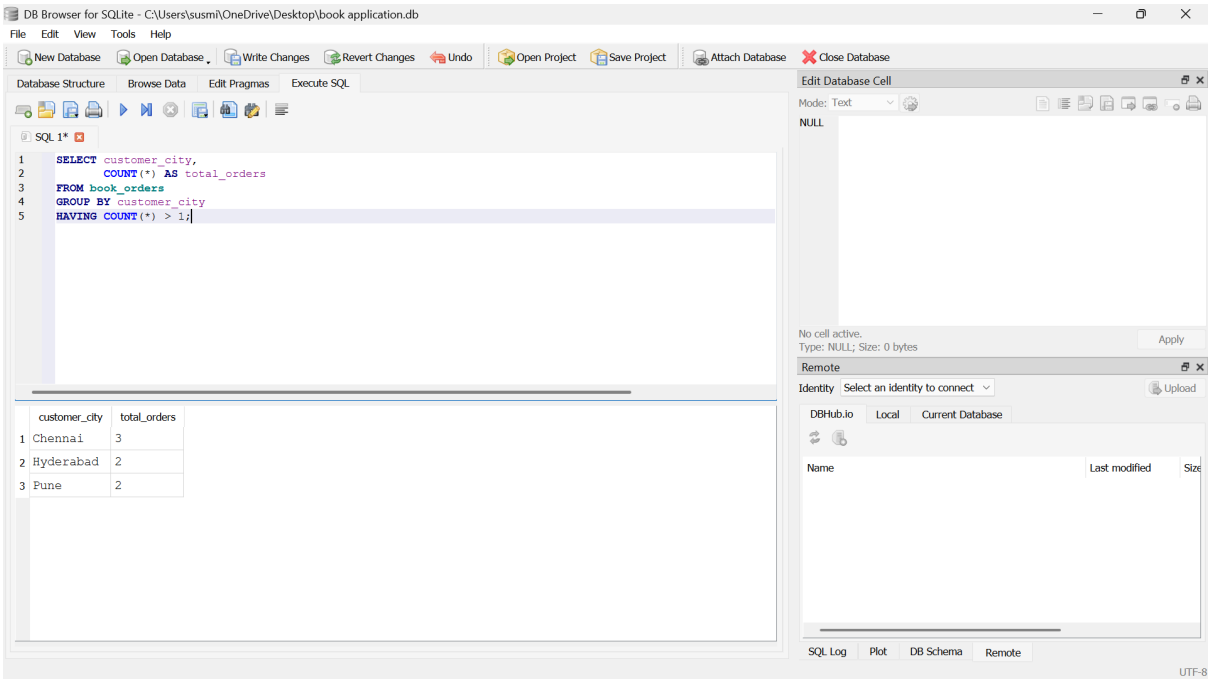
Step 6 – Count orders by payment method

```sql
SELECT payment_method,
       COUNT(*) AS total_orders
FROM book_orders
GROUP BY payment_method
ORDER BY total_orders DESC;
```



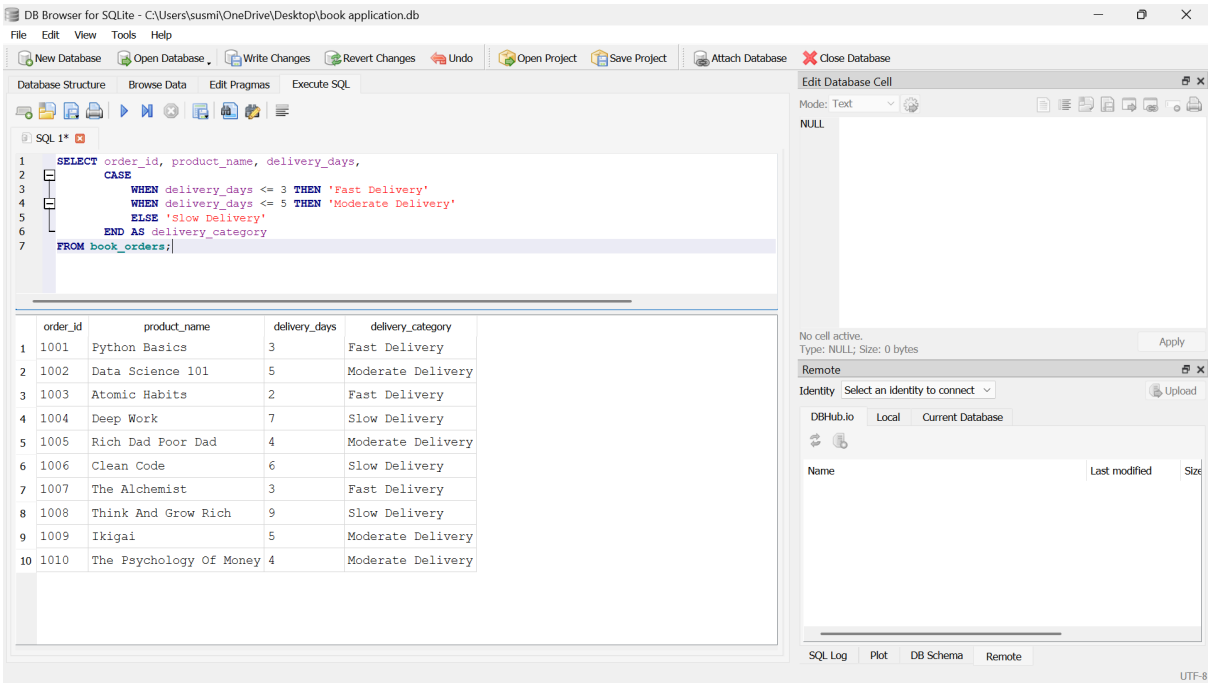
Step 7 – Find cities with more than one order

```sql
SELECT customer_city,
       COUNT(*) AS total_orders
FROM book_orders
GROUP BY customer_city
HAVING COUNT(*) > 1;
```



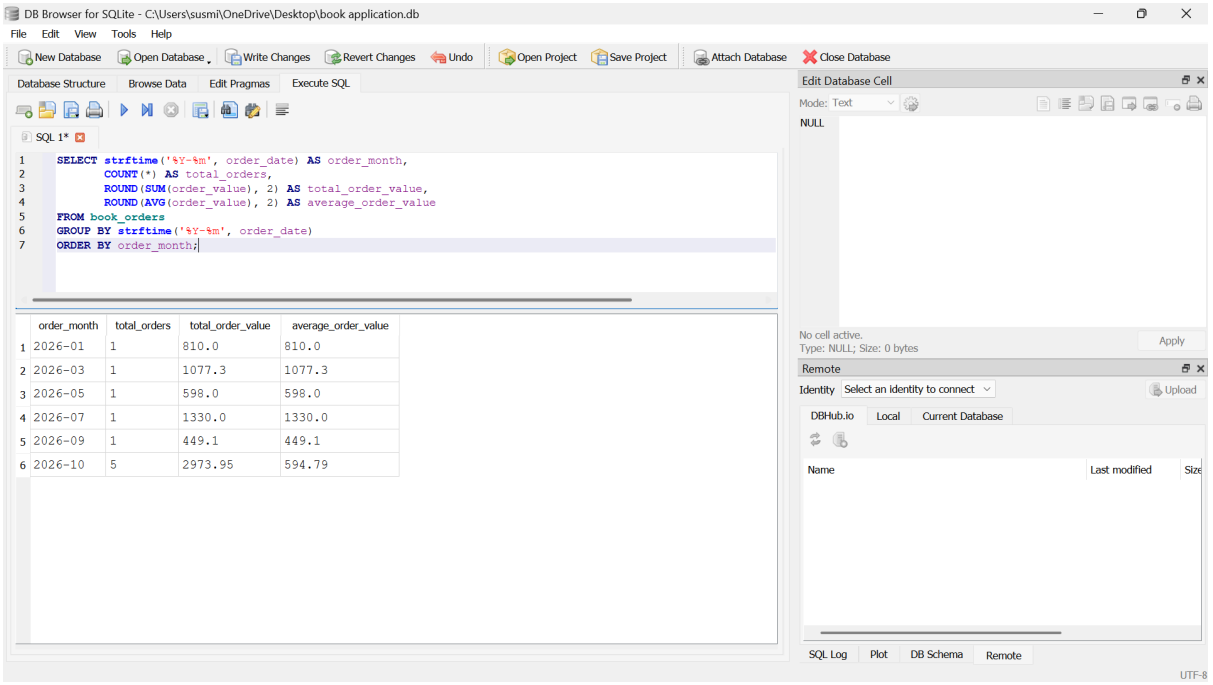
Step 8 – Classify orders by delivery time

```sql
SELECT order_id, product_name, delivery_days,
       CASE
           WHEN delivery_days <= 3 THEN 'Fast Delivery'
           WHEN delivery_days <= 5 THEN 'Moderate Delivery'
           ELSE 'Slow Delivery'
       END AS delivery_category
FROM book_orders;
```



Step 9 – Calculate monthly order statistics

```sql
SELECT strftime('%Y-%m', order_date) AS order_month,
       COUNT(*) AS total_orders,
       ROUND(SUM(order_value), 2) AS total_order_value,
       ROUND(AVG(order_value), 2) AS average_order_value
FROM book_orders
GROUP BY strftime('%Y-%m', order_date)
ORDER BY order_month;
```


DB Browser for SQLite - C:\Users\susmi\OneDrive\Desktop\book application.db
File Edit View Tools Help
New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database
Database Structure Browse Data Edit Pragmas Execute SQL
SQL 1*
1 SELECT strftime('%Y-%m', order_date) AS order_month,
2 COUNT(*) AS total_orders,
3 ROUND(SUM(order_value), 2) AS total_order_value,
4 ROUND(AVG(order_value), 2) AS average_order_value
5 FROM book_orders
6 GROUP BY strftime('%Y-%m', order_date)
7 ORDER BY order_month;
order_month total_orders total_order_value average_order_value
1 2026-01 1 810.0 810.0
2 2026-03 1 1077.3 1077.3
3 2026-05 1 598.0 598.0
4 2026-07 1 1330.0 1330.0
5 2026-09 1 449.1 449.1
6 2026-10 5 2973.95 594.79
Edit Database Cell
Mode: Text
NULL
No cell active.
Type: NULL; Size: 0 bytes
Apply
Remote
Identity Select an identity to connect
Upload
DBHub.io Local Current Database
Name Last modified Size
SQL Log Plot DB Schema Remote
UTF-8